

420-SN1-RE Programming in Science - Lab Exercise 5

Goals for this lab:

- Practice reading and writing `if` statements.
- Practice reading and writing `while` statements.

Part 1: Program tracing

Answer the questions about each of the following programs. You should answer first without using your computer (i.e., trace the program) and then use Python to check your answer. You are responsible for understanding all of these results, but you do not have to submit anything for this part.

1. What does the following program print?

```
x = 5
if x < 5:
    print("small")
elif x < 10:
    print("medium")
elif x < 20:
    print("large")
else:
    print("other")
```

2. How many times does this program print hello?

```
a = 0
b = 5
while a < b:
    if a % 2 == 0:
        print("hello")
    a += 1
b = 9
```

Part 2: Short programs

Exercise 1: Converting a numeric grade to a letter grade (lab05grade.py)

Create a program that converts a number grade to a letter grade.

- Ask the user to input a number grade.
- Print the corresponding letter grade for the user.
 - A grade between 90 and 100, inclusive, is an A.

- A grade greater than or equal to 80 but less than 90 is a B.
- A grade greater than or equal to 70 but less than 80 is a C.
- A grade greater than or equal to 60 but less than 70 is a D.
- A grade less than 60 is an F.

Examples:

- If the user inputs 73, your program should print Grade: C
- If the user inputs 60, your program should print Grade: D

Try to keep your program as short and simple as possible. For example, avoid unnecessary comparisons.

Hint: Consider the order of evaluation of the `if` statement in Part 1, problem 1.

Exercise 2: Shifty sum (lab05shift.py)

This is based on a problem from the 2017 Canadian Computing Competition.

Suppose you have a number like 12. Let's define "shifting a number" to mean adding a zero at the end. For example, if you shift 12 once, you get the number 120. If you shift 12 twice we get the number 1200. You can shift the number as many times as you want.

In this problem you will be calculating a "shifty sum", which is the sum of a number and the numbers you get by shifting. Specifically, you ask the user for the starting integer N and a non-negative integer k . You must add together N and all the numbers you get by shifting a total of k times. For example, the shifty sum when N is 12 and k is 1 is: $12 + 120 = 132$. As another example, the shifty sum when N is 12 and k is 3 is $12 + 120 + 1200 + 12000 = 13332$.

Write a program to compute the shifty sum. Prompt the user to enter N and then k , and print the resulting shifty sum.

Here's an example of the expected behaviour of the program:

```
Enter a value for N: 47
Enter a value for k: 3
The shifty sum is 52217
```

Exercise 3 - Tipsy turtle (lab05walk.py)

A random walk (also called a drunkard's walk) is a process in which a series of random steps are taken within a space. Random walks have been used in science, mathematics, and economics to model various processes such as the motion of gas particles (Brownian motion), animal foraging patterns, human decision-making, and the behavior of the stock market.

You will use Python and the `turtle` module to simulate and display a random walk in two dimensions for a given number of steps.

Program requirements and basic procedure:

1. Ask the user how many steps the simulation should take.
2. Use the `randint(n, m)` function from the `random` module and your knowledge of Python to generate (or roll) a random direction: either north, south, west, or east. (Note that the `randint(n, m)` function returns a random number between n and m *inclusive*).
3. Move the turtle some distance (your choice of a number of pixels) in that random direction.
4. Repeat steps 2 and 3 until the chosen number of steps have been taken.

5. Experiment with properties of the turtle interface (e.g., line width, line color, distance, etc.) so that the image is pleasing and the path the turtle has taken is clearly visible.
6. Consider modifying the speed of the turtle to make the simulation move faster.

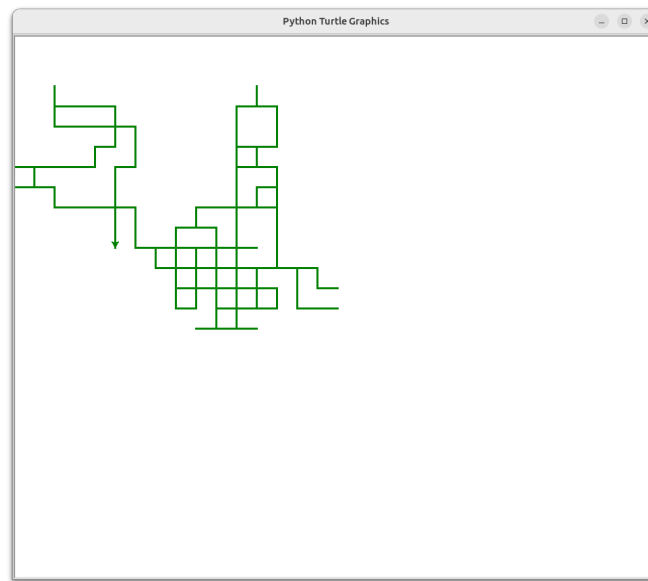


Figure 1: An example of a 200-step random walk with fixed distances of 30 pixels.

Submitting your work

Create a ZIP file containing your programs, and upload the ZIP file to Omnivox.

Turtle graphics reference

Here are a list of turtle functions that may be useful in this lab:

- `forward(x)` - move the turtle forward `x` pixels.
- `setheading(a)` - turn the turtle to face a particular direction given as an angle (`a`) from 0-360. 0 degrees faces east, 90 degrees faces north, 180 degrees faces west, and 270 degrees faces south.
- `goto(x, y)` - move the turtle to a specific point. The center of the drawing window is at 0, 0.
- `color(c)` - set the pen color. Most common colors can be referred to by `str` names, like `'blue'` or `'red'`.
- `up()` - raise the pen, temporarily stops drawing.
- `down()` - lower the pen, resumes drawing.
- `width(n)` - set the pen width to `n` pixels.
- `done()` - indicates that the drawing is complete. No more commands can be given after calling this function.
- `speed(s)` - set the drawing speed, where 1 is slow and 10 is fast. 0 is the fastest possible drawing speed.
- `exitonclick()` - keep the turtle window open until you click the window to exit.